IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

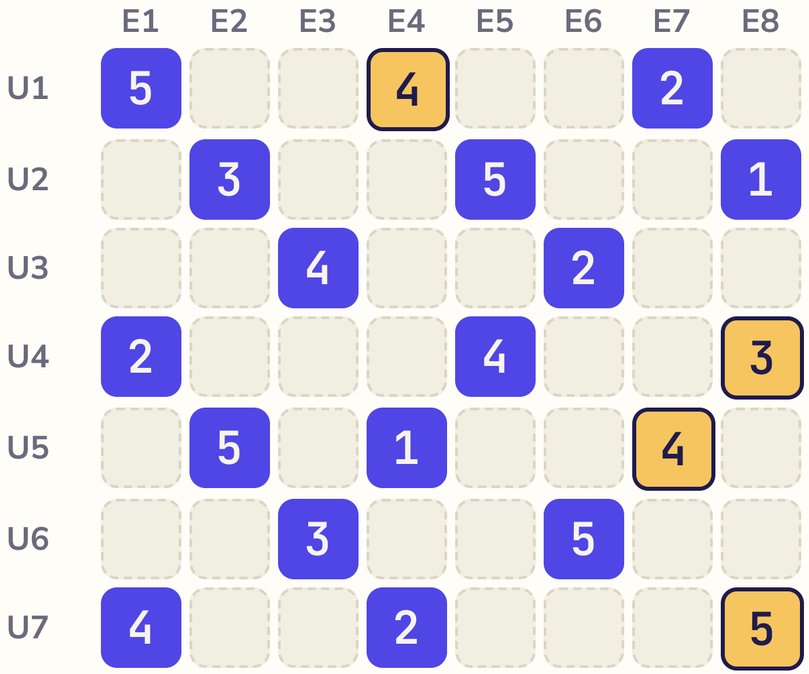
Recomendar experiencias a cada huésped

Teoría, práctica y, al final, material de consulta

Sprint 9 · S09_03 · teoría breve (25 min el 13/11), 50 min de práctica en clase (16/11) y material de consulta

LA IDEA

Una tabla casi vacía



Cada fila es un **usuario**, cada columna una **experiencia** y cada casilla, la nota de 1 a 5. Casi todas están **vacías**: nadie ha probado todo.

Es lo normal

En un recomendador real, **la mayoría de las casillas está vacía**, y justo esas son las que hay que adivinar.

Se esconde una parte de las notas (las doradas): el recomendador nunca las ve al aprender.

CÓMO SE EXAMINA

Esconder notas y medir cuánto se equivoca

nota escondida	real	predicción	error	error²
A	5	4	−1	1
B	4	4	0	0
C	2	4	+2	4
D	3	2	−1	1

1,22

RMSE del ejemplo: **√(6 ÷ 4)** estrellas

Punto de partida

Predecir siempre la **media global** da un RMSE que **cualquier recomendador debe mejorar**.

RMSE: la raíz de la media de los errores al cuadrado. Se mide en estrellas: menos es mejor.

MÉTODO 1 · POPULARIDAD

Lo mejor valorado, para todos por igual

Experiencia (ejemplo)	Nota media
Excursión A	4,5
Excursión B	4,0
Taller C	3,0

La misma lista para todos

Se recomienda lo **mejor valorado** de lo que el usuario **no ha probado**. No personaliza.

Arranque en frío

Es la única opción con un **huésped nuevo**, del que no se sabe nada.

Se calcula la media de cada experiencia **solo con el entrenamiento**, nunca con las notas escondidas.

MEDIR LA LISTA

precision@3: ¿acierta con la lista corta?

El cliente no ve notas: ve **3 sugerencias**. Se recomienda solo entre lo **no valorado** y se acierta si en la prueba le gustó (**4 estrellas o más**).

Usuario inventado	recomendada	¿le gustó?
Le gustaron: 3, 8 y 14	8	sí
	11	no
	20	no

precision@3 = 1 ÷ 3

De 3 recomendadas, le gustó **1**: **0,33**.

Solo usuarios evaluables

Cuentan los usuarios a los que **les gustó alguna** en la prueba. Un recomendador **al azar** sirve de referencia.

MÉTODO 2 · CONTENIDO

Si te gusta el mar, más experiencias de mar

El perfil de cada usuario es cuánto puntúa **por encima o por debajo de lo normal** cada categoría. Residuo = su nota – la nota media de esa experiencia.

Ejemplo inventado	valoraciones	suma de residuos	perfil = suma ÷ (n + 3)
mar	3	+3,5	0,58
aventura	1	+2,0	0,50
cultura	0	0	0,00

El «+3» es prudencia

Con **una sola** valoración no nos fiamos: sale 0,50 y no 2,0.

Descripción gruesa

Si solo se describe cada experiencia por su **categoría**, el método **no siempre mejora** a la popularidad.

MÉTODO 3 · COLABORATIVO

La gente como tú también valoró…

Sin describir las experiencias: dos experiencias son **parecidas** si la misma gente las puntúa por encima o por debajo de lo normal. La similitud es el **coseno** entre columnas de residuos.

usuario	exp. A	exp. B	exp. C
U1	+1	+1	−1
U2	−1	−1	+1
U3	+1	0	0
U4	0	+1	−1

A y B: **0,67** (van a la par) · B y C: **−1** (opuestas) · A y C: −0,67

Item-item

Compara **experiencias** (columnas): según quién las valoró.

User-user

Compara **usuarios** (filas): según qué valoraron.

Personalizan de verdad, pero con pocos datos por pareja las similitudes son bajas y ruidosas.

MÉTODO 4 · FACTORIZACIÓN

Gustos ocultos que se aprenden de los datos

Cada usuario tiene unos pocos **gustos ocultos** y cada experiencia una **intensidad** en cada uno. El algoritmo los aprende con las casillas conocidas y **rellena las vacías**.

$$\begin{pmatrix} 1{,}2 & -0{,}4 \end{pmatrix} \times \begin{pmatrix} 0{,}9 \\ 0{,}5 \end{pmatrix} = \begin{pmatrix} 0{,}88 \end{pmatrix}$$

gustos del usuario

intensidades de la experiencia

gustos · intensidades

Nota prevista (ejemplo)	
media global	2,97
+ sesgo del usuario (generoso)	+0,30
+ sesgo de la experiencia	+0,40
+ gustos · intensidades	+0,88
= nota prevista	4,55

k = cuántos gustos ocultos: pocos, modelo simple; demasiados, se ajusta al ruido.

PRÁCTICA · CUADERNO S09_03

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Cargar los datos, esconder el 20 % y construir la matriz	Clase
Ejercicio 1	Popularidad: la media de cada experiencia	Clase
Ejercicio 2	La función precision@k	Clase
Ejercicio 3	El perfil de gustos de cada usuario (contenido)	Clase
Ejercicio 4	Experiencias parecidas al Teide (item-item y user-user)	Casa
Ejercicio 5	La factorización y la tabla comparativa	Casa
Mini-reto	La función recomendar	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el ✔**. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

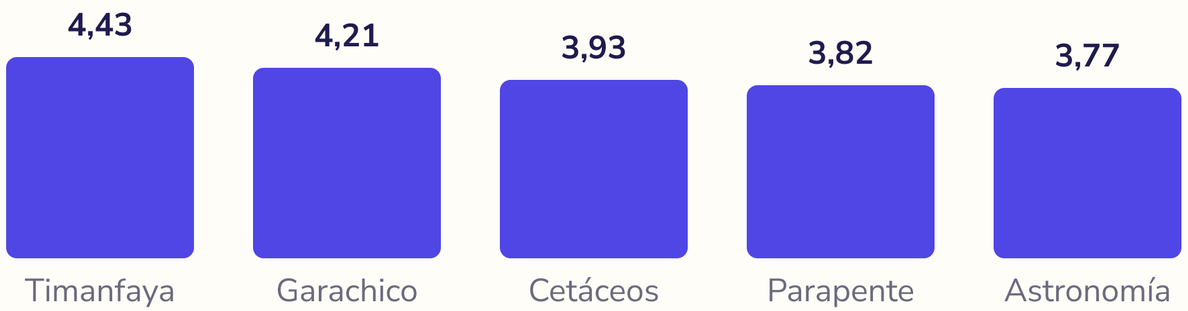
Cómo está ordenado

En el **mismo orden que el cuaderno**: ejercicios 1 a 5 y mini-reto. Cada uno trae su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	Popularidad	11 y 12
2	precision@k	13 y 14
3	Perfil de gustos (contenido)	15 y 16
4	Item-item y user-user	17 y 18
5	Factorización y comparación	19, 20 y 21
Mini-reto	La función recomendar	22 y 23

CONSULTA · EJERCICIO 1

Popularidad: media de cada experiencia



RMSE 1,029

Mejora la media global (1,275) sin personalizar nada.

Arranque en frío

Es la única opción con un **huésped nuevo**.

```
media_item = R.mean()                          # nota media de cada experiencia (solo entrenamiento)
P_pop = np.tile(media_item.values, (300, 1))   # la misma fila para los 300 usuarios
rmse_pop = evaluar_rmse(P_pop)                 # 1.029
```

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer la popularidad (ejercicio 1)

media_item = R.mean()

Con R, solo el **entrenamiento**. Con todas las valoraciones, el modelo «vería» las notas que debe adivinar.

np.tile(..., (300, 1))

Repite el vector de 25 medias en 300 filas: **la misma predicción para todos**.

1,275 frente a 1,029

Predecir siempre la media global (2,97) se equivoca **más de una estrella**. La media de cada experiencia baja a 1,029: no todas gustan igual.

Matriz y lista

R es 300 × 25 y solo el **24 %** tiene dato. Timanfaya (4,43) lidera la lista de popularidad, igual para todos: no personaliza.

CONSULTA · EJERCICIO 2

La función precision@3 (ejercicio 2)

El cliente no ve notas: ve **3 sugerencias**. Se recomienda solo entre lo **no valorado** y se acierta si en la prueba le gustó (**4 estrellas o más**).

Usuario inventado	recomendada	¿le gustó?
Le gustaron: 3, 8 y 14	8	sí
	11	no
	20	no

precision@3 = 1 ÷ 3

De 3 recomendadas, le gustó **1**: **0,33**.

popularidad **0,125** · azar **0,081** · **115** usuarios evaluables

```
puntuacion = P[u - 1].astype(float).copy()
puntuacion[M[u - 1]] = -np.inf                    # oculta lo que ya valoró
recomendadas = np.argsort(-puntuacion)[:k] + 1    # las k de mayor puntuación
```

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer precision@3 (ejercicio 2)

puntuacion[M[u - 1]] = -np.inf

Las ya valoradas quedan **al final** del orden: no se recomienda lo que el usuario conoce.

argsort(-puntuacion)[:k] + 1

argsort da posiciones desde 0: se **suma 1** para pasar a id_experiencia.

115 usuarios evaluables

No son 300: solo cuentan quienes, en la prueba, dieron **4 o 5 estrellas** a alguna experiencia.

0,125 frente a 0,081

precision@3 de la popularidad frente a recomendar **al azar**: mejora, pero poco. Si no baja de lo que da el azar, no aporta.

CONSULTA · EJERCICIO 3

El perfil de gustos de cada usuario (ejercicio 3)

El perfil de cada usuario es cuánto puntúa **por encima o por debajo de lo normal** cada categoría. Residuo = su nota − la nota media de esa experiencia.

Ejemplo inventado	valoraciones	suma de residuos	perfil = suma ÷ (n + 3)
mar	3	+3,5	0,58
aventura	1	+2,0	0,50
cultura	0	0	0,00

El «+3» es prudencia

Con **una sola** valoración no nos fiamos: sale 0,50 y no 2,0.

RMSE 1,066 · precision@3 0,130

Peor que la popularidad en RMSE (1,029).

```
perfil_usuario = (C @ F) / (M.astype(float) @ F + 3)     # 300 usuarios × 7 categorías
P_cont = media_item_v[None, :] + perfil_usuario @ F.T    # media + afinidad
```

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer el perfil de gustos (ejercicio 3)

(C @ F) / (M @ F + 3)

F (25 × 7) marca la categoría de cada experiencia; **C**, los residuos. Da el perfil de **300 × 7**.

Por qué el + 3

Prudencia: con pocas valoraciones el perfil se queda **cerca de 0**. Sin valoraciones: todo 0, y se evita dividir 0 entre 0.

+0,34 en **mar**

Ese usuario valora el mar **0,34 estrellas por encima** de su media. Residuo de un 5 a una experiencia de media 4,0: **+1,0**.

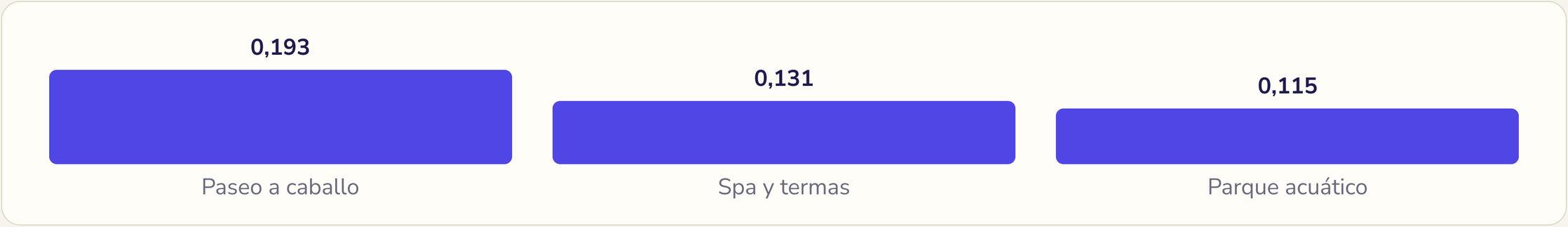
Contenido frente a popularidad

RMSE **1,066** (peor que 1,029) y precision@3 0,13 (0,125). 7 categorías para 25 experiencias son una descripción **muy gruesa**.

CONSULTA · EJERCICIO 4

Experiencias parecidas al Teide (ejercicio 4)

```
S = cosine_similarity(C.T)
np.fill_diagonal(S, 0)                                  # la diagonal a 0
ids_similares = np.argsort(-S[0])[:3] + 1
similares_teide = (experiencias.set_index("id_experiencia")
                   .loc[ids_similares, "nombre"].tolist())    # 3 nombres
```



item-item: RMSE 0,895 · precision@3 0,171. user-user: RMSE 0,865 · precision@3 0,148.

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer las similitudes (ejercicio 4)

cosine_similarity(C.T)

C.T compara **experiencias** (25 × 25). Con C serían usuarios (300 × 300).

np.fill_diagonal(S, 0)

Una experiencia consigo misma da 1 y **siempre sería la más parecida**: se quiere el resto.

Similitudes bajas

0,19 como máximo: las valoraciones de unas **dicen poco** de otras. Hay mucho ruido y pocos datos por pareja (**76 %** vacío).

Item-item y user-user

Ambos **baten a la popularidad** (1,029): 0,895 y 0,865. User-user acierta mejor la nota; item-item, la lista (0,171).

CONSULTA · EJERCICIO 5

La factorización (ejercicio 5)

```
P_mf = factorizar(R, k=3)
rmse_mf = evaluar_rmse(P_mf)    # 0.860
p_mf = precision_k(P_mf)        # 0.180
```

0,860
RMSE

0,180
precision@3

gustos ocultos k	2	3	4	6
RMSE	0,875	**0,860**	0,867	0,870

CONSULTA · EJERCICIO 5 · COMPARACIÓN

Cambiar de métrica cambia el orden

Método	Qué mira	RMSE (menos es mejor)	precision@3 (más es mejor)
Media global	nada	1,275	—
Popularidad	lo mejor de todos	1,029	0,125
Contenido	la categoría	1,066	0,130
Item-item	experiencias parecidas	0,895	0,171
User-user	usuarios parecidos	0,865	0,148
Factorización (k = 3)	gustos ocultos	0,860	0,180

El orden cambia

Contenido pierde con popularidad en RMSE pero la supera en precision@3. User-user es 2.º en RMSE y 3.º en precision@3.

115 usuarios evaluables

Una diferencia de **0,01** en precision@3 es **ruido**: no se la vendas al cliente.

CONSULTA · ACLARACIONES · EJERCICIO 5

Leer la comparación (ejercicio 5)

factorizar(R, k=3)

Aprende **3 gustos ocultos** por usuario y por experiencia: nadie los etiqueta. Con más (6) empeora un poco: el modelo **se ajusta al ruido**.

Elegir k

Aquí se compara **k** mirando la prueba. En un caso real, con una **validación aparte**.

Quién gana y quién pierde

Factorización gana en RMSE (0,860) y precision@3 (0,180). **Contenido** pierde (1,066), peor que popularidad. **Tres** métodos baten a popularidad.

La métrica cambia el orden

El 2.º en RMSE es user-user (0,865), en precision@3, item-item (0,171). Con ~115 usuarios, **0,01** de diferencia es **ruido**.

CONSULTA · MINI-RETO

La función recomendar (mini-reto)

```
def recomendar(id_usuario, n=3, P=P_mf):
    if not 1 <= id_usuario <= len(M):
        raise ValueError(...)
    puntuacion = P[id_usuario - 1].astype(float).copy()
    puntuacion[M[id_usuario - 1]] = -np.inf
    ids = np.argsort(-puntuacion)[:n] + 1
```

usuario	experiencia	nota prevista
1	Parque Timanfaya	4,69
1	Ruta por Anaga	4,34
1	Zoo y jardines	3,50
2	Observación de cetáceos	4,39
2	Astronomía en el Roque	4,19
2	Visita a Garachico	3,96

Para el reto

Allí se compara con la popularidad el método **item-item**: **recomendar(..., P=P_item)**.

Sesgo de popularidad

Con popularidad, las listas de los 300 usuarios usan solo **8 de las 25** experiencias; con item-item, **18**.

CONSULTA · ACLARACIONES · MINI-RETO

Leer la función recomendar (mini-reto, casa)

raise ValueError(...)

`recomendar(0)` usaría `P[-1]`: **el último usuario**, sin avisar. Es mejor fallar que recomendar a otra persona.

-inf y [:n] + 1

Oculta lo ya valorado, ordena y suma 1 para pasar a `id_experiencia`.

Dos usuarios, dos listas

Misma función, **distinta fila de predicciones** y distintas experiencias ya valoradas. Usuario 1: Timanfaya (4,69).

Para el reto

Se compara con popularidad el método **item-item**: `recomendar(..., P=P_item)`. Con popularidad, las listas usan **8 de 25** experiencias; con item-item, **18**.